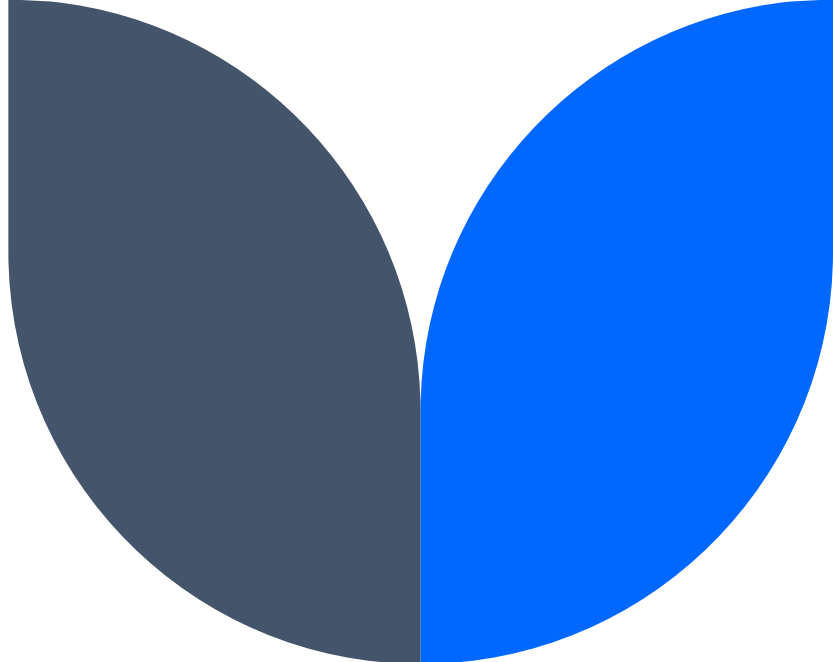
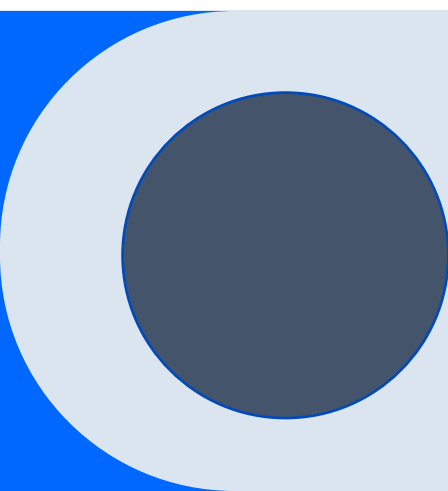




Contract Monthly Claim System



**PROG6212
ST10445500**

Project Overview and Purpose

- Previous Feedback
- Project Overview
- Core Requirements
- System Architecture
- Key Technologies
- Important Links

Previous Feedback and Reflection

I received critical feedback from my part 2 submission which I have prioritised when developing my final submission for the PROG6212 POE.

I was missing an entire view, as I seemed to have misinterpreted the needs of the system. I had combined both the Programme Coordinator and Academic Manager Views into one called Admin. This was wrong, as it took away the functionality of verification of claims BEFORE approval.

To reflect and integrate this feedback into part 3, I have made the following changes:

- I have made the separate views for the Programme Coordinator and Academic Manager. Additionally, a login page and additional HR view has been made for new requirements.
- Finally, I spent more time on development of part 3 of the POE, as family issues have since calmed down since the increase in around the time part 2 was due.



Application Purpose

A monthly claim system for part-time lecturers ensures that all working hours are accurately recorded and submitted for payment each month.

It provides lecturers with a clear, organised way to log their work while allowing administrative staff to efficiently review, verify, and approve claims.

This system improves transparency, reduces errors, streamlines payroll processing, and ensures lecturers are compensated on time.

For administrators, it centralises claim management, supports accountability, and maintains proper institutional records for auditing and budgeting purposes.

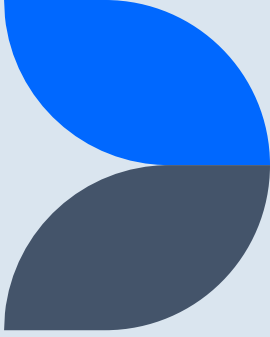


Updated Controllers and Views

```
Views
├── Account
│   └── Login.cshtml
├── Coordinator
│   └── Index.cshtml
├── Home
│   ├── Index.cshtml
│   └── Privacy.cshtml
├── HR
│   ├── ApprovedClaimsReport.cshtml
│   ├── CreateUser.cshtml
│   ├── Index.cshtml
│   └── ViewAllUsers.cshtml
├── Lecturer
│   ├── Index.cshtml
│   └── SubmitClaim.cshtml
├── Manager
│   └── Index.cshtml
└── Shared
    ├── _Layout.cshtml
    ├── _ValidationScriptsPartial.cshtml
    └── Error.cshtml
```

```
Controllers
├── Api
│   └── ClaimsApiController.cs
├── AccountController.cs
├── BaseController.cs
├── CoordinatorController.cs
├── HomeController.cs
├── HRController.cs
├── LecturerController.cs
└── ManagerController.cs
```

HR Feature and Functionality



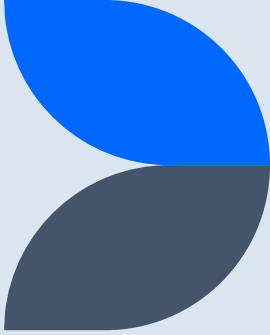
HR is responsible for creating all user accounts (this must be implemented as part of the UI view): Lecturers, Coordinators, Managers.

HR will act as a system admin, therefore HR accounts cannot be created. When HR creates a lecturer account, their hourly rate will be defined.

Implementation:

- A login feature was created, with the initial seeding of an HR account.
- From there, all roles are able to log in and out with their detailed credentials.
- HR can create new users (staff) and fill out information needed.
- They can select one of three roles provided.
- HR defines the hourly rate for lecturers upon user creation.

Login Page



CMCS Claims System

 Guest

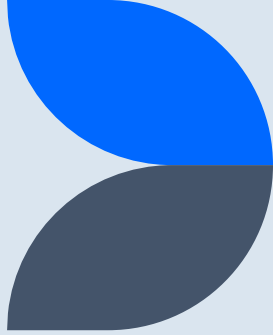
CMCS Claims System

Email

Password

Login

HR Dashboard



CMCS Claims System

 HR Administrator **HR**

Logout

HR Dashboard

Welcome, HR Administrator



Create New User

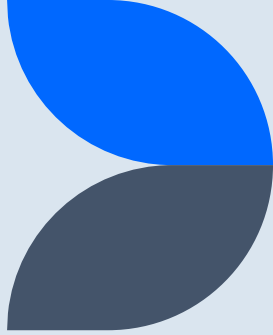


View Approved Claims



View All Users

HR: Create New User



CMCS Claims System



HR Administrator

HR

Logout

Create New User

Name

Email

Password

Role

Hourly Rate (Lecturer only)

Create User

Back

HR: View Approved Claims

CMCS Claims System

 HR Administrator

HR

Logout

Approved Claims Report

Download PDF Report

Back to Dashboard

Lecturer	Hours	Rate	Total	Submitted
John Smith	15,00	R 550,00:F2	R 8250,0000:F2	2025/11/21 04:18:57:yyyy-MM-dd

HR: View All Users

CMCS Claims System

 HR Administrator

HR

Logout

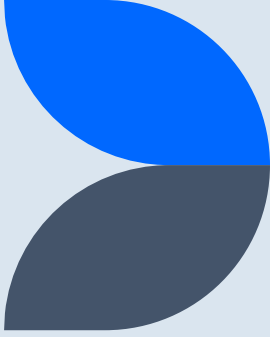
All Users in System

Back to HR Dashboard

Total Users: 5

Name	Email	Role	Hourly Rate
John Smith	john@cmcs.ac.za	Lecturer	R 550,00
Sarah Williams	sarah@cmcs.ac.za	Lecturer	R 620,00
Michael Brown	coord@cmcs.ac.za	Coordinator	— N/A —
Dr. Amanda Johnson	manager@cmcs.ac.za	Manager	— N/A —
HR Administrator	hr@cmcs.ac.za	HR	— N/A —

Authentication and Authorisation



Sessions must be used for authentication & authorisation. I.e., you must make of Sessions to ensure users cannot access pages they are not supposed to access.

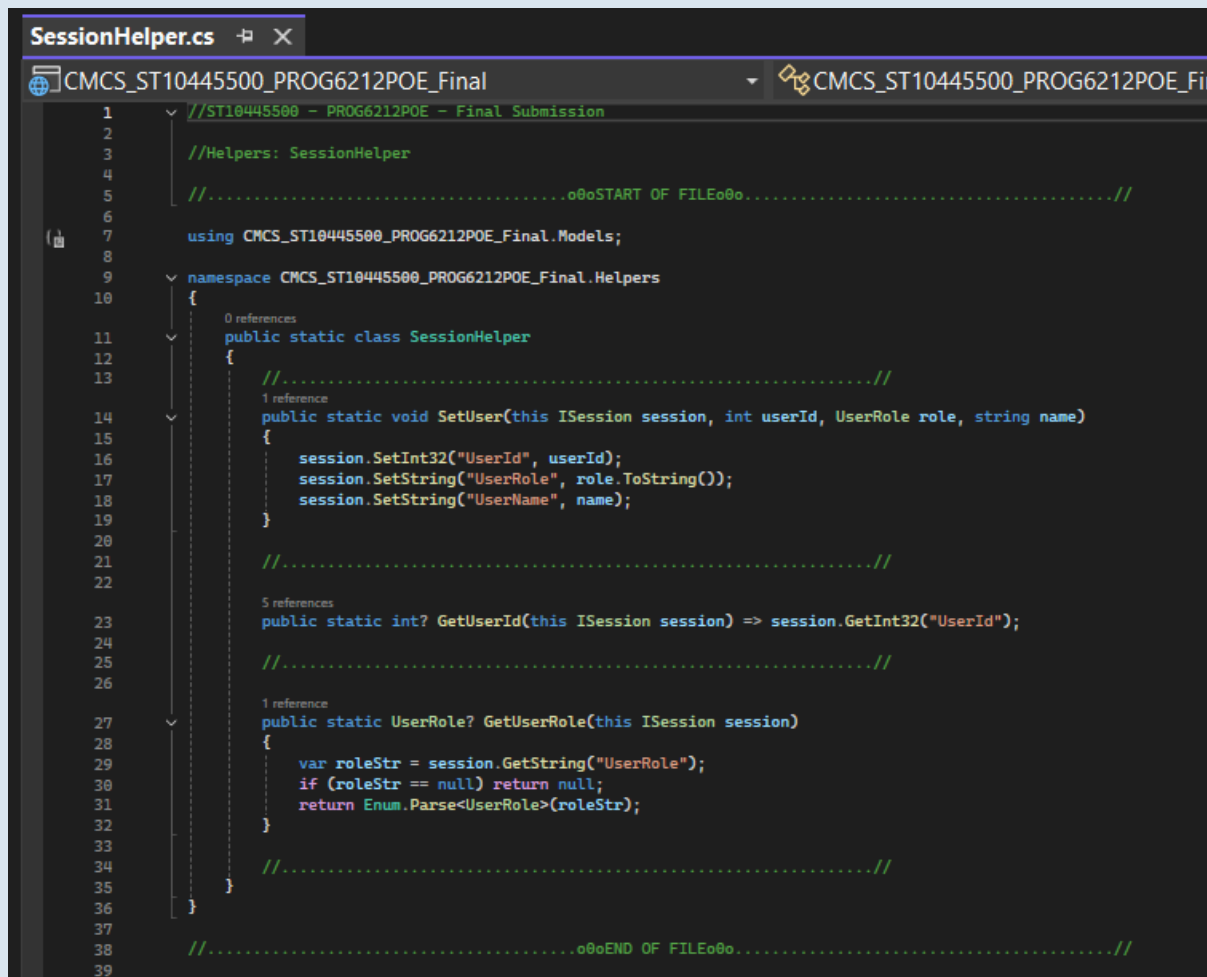
For example, Lecturers should not see what a Coordinator sees; Managers should not see what Coordinators see; None of the above should see what HR can see. All of this must be based on the type of user that has logged in (role/Id).

ASP.NET Identity should NOT be used (Sessions instead). If you choose to use Identity, please make sure they are custom.

Implementation:

- Sessions have been used throughout the project to maintain role access limitations.
- Helpers/SessionHelper.cs made.

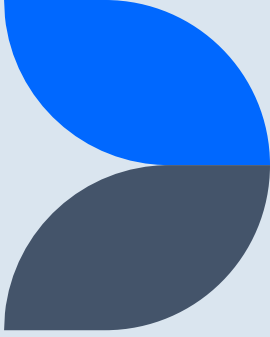
Helpers/SessionHelper.cs



The image shows a Visual Studio code editor window with the file 'SessionHelper.cs' open. The editor has a dark theme. The file is part of a project named 'CMCS_ST10445500_PROG6212POE_Final'. The code is in C# and defines a static class 'SessionHelper' within the namespace 'CMCS_ST10445500_PROG6212POE_Final.Helpers'. The class contains three methods: 'SetUser', 'GetUserId', and 'GetUserRole'. The 'SetUser' method takes an 'ISession' object, a user ID, a user role, and a name, and sets these values in the session. The 'GetUserId' method returns the user ID from the session. The 'GetUserRole' method returns the user role from the session, parsing it from a string. The code is surrounded by file headers and footers. The editor interface includes a sidebar on the left with a file explorer, a top toolbar with icons for file operations, and a bottom status bar.

```
1 //ST10445500 - PROG6212POE - Final Submission
2
3 //Helpers: SessionHelper
4
5 //.....o0oSTART OF FILEo0o.....//
6
7 using CMCS_ST10445500_PROG6212POE_Final.Models;
8
9 namespace CMCS_ST10445500_PROG6212POE_Final.Helpers
10 {
11     0 references
12     public static class SessionHelper
13     {
14         //.....//
15         1 reference
16         public static void SetUser(this ISession session, int userId, UserRole role, string name)
17         {
18             session.SetInt32("UserId", userId);
19             session.SetString("UserRole", role.ToString());
20             session.SetString("UserName", name);
21         }
22         //.....//
23         5 references
24         public static int? GetUserId(this ISession session) => session.GetInt32("UserId");
25         //.....//
26         1 reference
27         public static UserRole? GetUserRole(this ISession session)
28         {
29             var roleStr = session.GetString("UserRole");
30             if (roleStr == null) return null;
31             return Enum.Parse<UserRole>(roleStr);
32         }
33         //.....//
34     }
35 }
36
37
38 //.....o0oEND OF FILEo0o.....//
39
```

Controllers/AccountController.cs



```
[HttpPost]
0 references
public async Task<IActionResult> Login(string email, string password)
{
    var user = _db.Users.FirstOrDefault(u => u.Email == email);
    if (user != null && BCrypt.Net.BCrypt.Verify(password, user.PasswordHash))
    {
        HttpContext.Session.SetUser(user.Id, user.Role, user.Name);

        return user.Role switch
        {
            UserRole.Lecturer => RedirectToAction("Index", "Lecturer"),
            UserRole.Coordinator => RedirectToAction("Index", "Coordinator"),
            UserRole.Manager => RedirectToAction("Index", "Manager"),
            UserRole.HR => RedirectToAction("Index", "HR"),
            _ => RedirectToAction("Login")
        };
    }

    ViewBag.Error = "Invalid credentials";
    return View();
}
```

Role
authentication
in login.

Controllers/BaseController.cs

```
10 //.....o0oSTART OF FILEo0o.....//
11
12 namespace CMCS_ST10445500_PROG6212POE_Final.Controllers
13 {
14     7 references
15     public class BaseController : Controller
16     {
17         protected readonly AppDbContext _db;
18         3 references
19         public BaseController(AppDbContext db) => _db = db;
20
21         //.....//
22         6 references
23         protected IActionResult RequireLogin()
24         {
25             if (HttpContext.Session.GetUserId() == null)
26                 return RedirectToAction("Login", "Account");
27             return null!;
28         }
29
30         //.....//
31         6 references
32         protected IActionResult RequireRole(UserRole requiredRole)
33         {
34             var role = HttpContext.Session.GetUserRole();
35             if (role != requiredRole)
36                 return RedirectToAction("Login", "Account");
37             return null!;
38         }
39     }
40
41 //.....o0oEND OF FILEo0o.....//
```

Controllers/LecturerController.cs

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> SubmitClaim(decimal hoursWorked, string? notes)
{
    var check = RequireLogin(); if (check != null) return check;
    if (RequireRole(UserRole.Lecturer) != null) return RedirectToAction("Login", "Account");

    if (hoursWorked <= 0)
        ModelState.AddModelError("hoursWorked", "Hours must be greater than 0.");

    if (!ModelState.IsValid)
    {
        //re-load the rate again if validation fails
        var userId = HttpContext.Session.GetUserId().Value;
        var user = await _db.Users.FindAsync(userId);
        ViewBag.HourlyRate = user?.HourlyRate ?? 0m;
        return View();
    }

    var userId2 = HttpContext.Session.GetUserId().Value;
    var lecturer = await _db.Users.FindAsync(userId2);

    var claim = new Claim
    {
        LecturerId = userId2,
        HoursWorked = hoursWorked,
        HourlyRate = lecturer!.HourlyRate ?? 0m,
        Notes = notes,
        SubmittedDate = DateTime.Now,
        Status = ClaimStatus.Pending
    };

    _db.Claims.Add(claim);
    await _db.SaveChangesAsync();

    return RedirectToAction("Index");
}
```

Lecturers can no longer define their Hourly Rate and can only be done so through HR.

Only lecturers can submit claims and can see the views needed for that functionality.

Services/ReportGenerator.cs

```
1 using QuestPDF.Fluent;
2 using QuestPDF.Helpers;
3 using QuestPDF.Infrastructure;
4 using CMCS_ST10445500_PROG6212POE_Final.Models;
5
6 //ST10445500 - PROG6212POE - Final Submission
7
8 //Services: ReportGenerator
9
10 //WARNING! This section of code was generated by ChatGPT and edited accordingly.
11 //PS: I had no idea how do to create these reports, so I had to use AI to assist me with general structure and I tweaked it to fit my needs.
12
13 //.....o0oSTART OF FILEo0o.....//
14
15 namespace CMCS_ST10445500_PROG6212POE_Final.Services
16 {
17     1 reference
18     public static class ReportGenerator
19     {
20         1 reference
21         public static IDocument GenerateApprovedClaimsReport(List<Claim> claims)
22         {
23             return Document.Create(container =>
24             {
25                 container.Page(page =>
26                 {
27                     page.Size(PageSizes.A4);
28                     page.Margin(2, Unit.Centimetre);
29                     page.PageColor(Colors.White);
30
31                     // === HEADER ===
32                     page.Header()
33                         .AlignCenter()
34                         .Text("CMCS APPROVED CLAIMS REPORT")
35                         .FontSize(24)
36                         .Bold()
37                         .FontColor("#005EB8");
38
39                 });
40             });
41         }
42     }
43 }
44
45 [HttpGet]
46 0 references
47 public IActionResult DownloadReportPdf()
48 {
49     var claims = _db.Claims
50         .Include(c => c.Lecturer)
51         .Where(c => c.Status == ClaimStatus.Approved)
52         .ToList();
53
54     var document = ReportGenerator.GenerateApprovedClaimsReport(claims);
55     var pdfBytes = document.GeneratePdf();
56
57     return File(pdfBytes, "application/pdf", $"Approved_Claims_Report_{DateTime.Now:yyyyMMdd}.pdf");
58 }
```

Uses QuestPDF
downloaded via NuGet
package.

Called in
Controllers/HRController.
cs.

Swagger



Select a definition **CMCS Claims API V1**

CMCS Claims API ^{V1} ^{OAS 3.0}

/swagger/v1/swagger.json

ClaimsApi

GET	/api/claims/my-claims	▼
POST	/api/claims/submit	▼
GET	/api/claims/pending	▼
POST	/api/claims/verify/{claimId}	▼
POST	/api/claims/reject/{claimId}	▼
GET	/api/claims/verified	▼
POST	/api/claims/approve/{claimId}	▼
POST	/api/claims/reject-manager/{claimId}	▼

```
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

//add Swagger
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new() { Title = "CMCS Claims API", Version = "v1" });
});

//register DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

//.....//

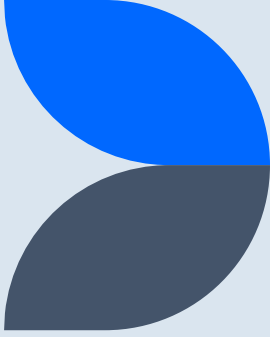
var app = builder.Build();

//middleware
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
else
{
    app.UseSwagger();
    app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "CMCS Claims API V1"));
}
```

Key Technologies Used

1. .NET 8 MVC
2. Entity Framework Core + LocalDB
3. Sessions (custom auth)
4. QuestPDF for reports
5. Swashbuckle (Swagger)
6. Bootstrap 5 + custom blue theme

Role Login Information Needed



Email	Password	Name	Role
hr@cmcs.ac.za	Hr@2025	HR Administrator	HR
john@cmcs.ac.za	Lect123!	John Smith	Lecturer
sarah@cmcs.ac.za	Sarah456!	Sarah Williams	Lecturer
coord@cmcs.ac.za	Coord789!	Michael Brown	Coordinator
manager@cmcs.ac.za	Manager101!	Dr. Amanda Johnson	Academic Manager

Here is the critical login information needed when running the project.

It is also listed on my README.

GitHub Attached in Submission

The repository includes additional information such as the following presentation, the link to the YouTube video made, an updated README, and important information regarding login details for the various roles.





Thank you!

PROG6212

ST10445500

BCAD Year 2 – Group 1

ST10445500@vcconnect.edu.za